

ML modelling

ML.NET is used for identifying patterns, predictive analytics, feature extraction and real-time data analysis. The *imdb_labelled.txt* can be downloaded from <https://archive.ics.uci.edu/ml/machine-learning-databases/00331/sentiment%20labelled%20sentences.zip>.

The data snippet is shown below:

A very, very, very slow-moving, aimless movie about a distressed, drifting young man. 0
Not sure who was more lost - the flat characters or the audience, nearly half of whom walked out. 0
Attempting artiness with black & white and clever camera angles, the movie disappointed - became even more ridiculous - as the acting was poor and the plot and lines almost non-existent. 0
Very little music or anything to speak of. 0
The best scene in the movie was when Gerardo is trying to find a song that keeps running through his head. 1
The rest of the movie lacks art, charm, meaning... If it's about emptiness, it works I guess because it's empty. 0
Wasted two hours. 0

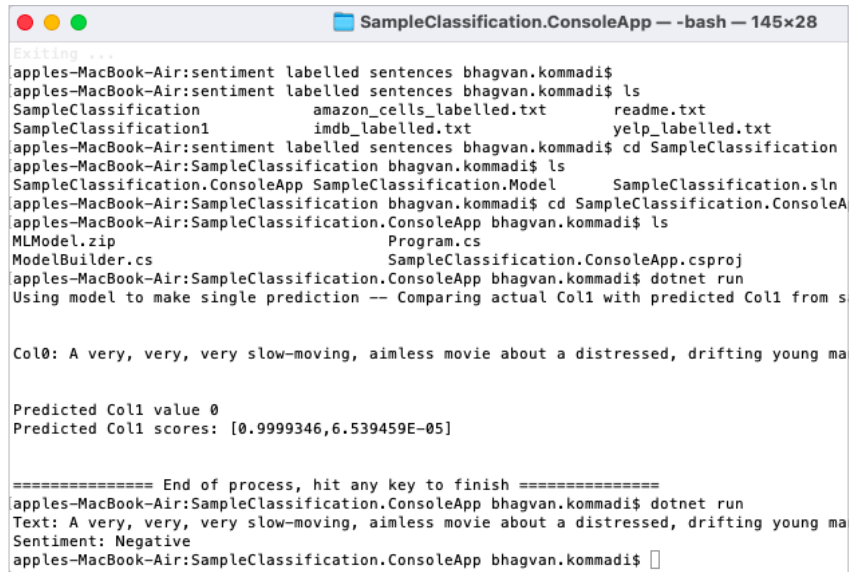
The command used in ML.NET for classification is mentioned below, and Figure 3 shows the output.

```
mlnet classification --dataset "imdb_labelled.txt" --has-header false
--label-col 1 --train-time 60
```

The code that is generated as an output will have the following:

1. SampleClassification folder
2. A .NET console app (SampleClassification.ConsoleApp) having a ModelBuilder.cs and a Program.cs.
3. A .NET Standard class library (SampleClassification.Model) has a ModelInput.cs, ModelOutput.cs, and MLModel.zip.

The console app can be executed by



```

apples-MacBook-Air:sentiment labelled sentences bhagvan.kommadi$
apples-MacBook-Air:sentiment labelled sentences bhagvan.kommadi$ ls
SampleClassification      amazon_cells_labelled.txt  readme.txt
SampleClassification1    imdb_labelled.txt          yelp_labelled.txt
apples-MacBook-Air:sentiment labelled sentences bhagvan.kommadi$ cd SampleClassification
apples-MacBook-Air:SampleClassification bhagvan.kommadi$ ls
SampleClassification.ConsoleApp SampleClassification.Model SampleClassification.sln
apples-MacBook-Air:SampleClassification bhagvan.kommadi$ cd SampleClassification.ConsoleApp
apples-MacBook-Air:SampleClassification.ConsoleApp bhagvan.kommadi$ ls
MLModel.zip                Program.cs
ModelBuilder.cs            SampleClassification.ConsoleApp.csproj
apples-MacBook-Air:SampleClassification.ConsoleApp bhagvan.kommadi$ dotnet run
Using model to make single prediction -- Comparing actual Col1 with predicted Col1 from s

Col0: A very, very, very slow-moving, aimless movie about a distressed, drifting young ma

Predicted Col1 value 0
Predicted Col1 scores: [0.9999346,6.539459E-05]

===== End of process, hit any key to finish =====
apples-MacBook-Air:SampleClassification.ConsoleApp bhagvan.kommadi$ dotnet run
Text: A very, very, very slow-moving, aimless movie about a distressed, drifting young ma
Sentiment: Negative
apples-MacBook-Air:SampleClassification.ConsoleApp bhagvan.kommadi$

```

Figure 3: Sample classification after execution of *dotnet run*

copying MLModel.zip from the *Model* folder to the *ConsoleApp* folder. This app gives the sentiment of the input sentence. The command for execution is:

```
dotnet run
```

ML training

Model training happens when the ML.NET lazy loading technique starts. This technique starts after calling the *Fit()* method. The estimator helps in creating a transformer after learning, while the input is the training data and the output of the transformer is the trained model.

ML testing

Machine learning data is split into training, testing, and validation data sets. The tests are done to validate the data. There are different test types, as mentioned below:

- Invariance
- Directional expectation
- Minimum functionality

ML evaluation

You can collect the following metrics for the model using ML.NET:

- Accuracy
- Area under the curve (AUC)
- R-Squared
- Root Mean Squared Error (RMSE)
- Recall
- Precision
- Confusion matrix
- Learning
- Regularisation
- Batch size
- Depth of layers

ML deployment

Code generated from the ML.NET CLI is checked into Git (source code management tool) and can be deployed on Azure or any other cloud with Windows OS. DevOps tools like Jenkins can be used to deploy the code and data from the Git repository.

ML.NET helps in creating an ML model that can be updated frequently to provide real-time predictions and hence the business value. **END** 🐧

By: Bhagvan Kommadi

The author is founder of 'Quantica Computacao', India's first quantum computing startup. He has 20 years of experience in product development and IT services.